EAP-EDHOC-PQC: Quantum-Safe Onboarding for IoT E-Business Based on Extensible Authentication Protocol*

Adi Panca Saputra Iskandar[1,2], I Wayan Adi Juliawan Pawana[1,2], and Ilsun You[2,3†]

[1] Department of Cybersecurity Kookmin University, Seoul, South Korea
{adipanca, adijuliawanpawana, isyou}@kookmin.ac.kr
[2] Department of Electrical Engineering Udayana University, Bali, Indonesia
{adipanca, adijuliawanpawana}@unud.ac.id
[3] KMU Global Research Center for ICT Convergence Security, Kookmin University
ilsunu@gmail.com

Abstract

The Internet of Things (IoT) underpins a growing range of e-business operations networked vending machines, smart point-of-sale and self-checkout terminals, electronic shelf labels, smart-retail beacons, electric-vehicle (EV) chargers, smart parking meters, fleet and cold-chain trackers, and industrial sensor gateways all of which onboard onto Wi-Fi or other non-3GPP networks via the Extensible Authentication Protocol (EAP) and must remain secure against a future quantum adversary. Ephemeral Diffie–Hellman over COSE (EDHOC, RFC 9528) is a lightweight authenticated key exchange for constrained devices, and the Internet-Draft `draft-papon-lake-pq-edhoc` defines five post-quantum (PQ) authentication variants over ML-KEM and ML-DSA. The combined impact of PQ key enlargement, EAP fragmentation, and RADIUS transport on a real AAA deployment, however, has not yet been quantified, and formal analyses of the PQ-EDHOC variants integrated with EAP are scarce. We close these gaps by proposing *EAP-EDHOC-PQ*, a unified EAP method that embeds all five PQ-EDHOC variants. We verify mutual authentication, session-key secrecy, forward secrecy, and replay resistance under a Dolev–Yao adversary in ProVerif, and we empirically evaluate the design on a heterogeneous IoT testbed (Raspberry Pi 5 Initiator, MacBook Air M3 Authenticator/Responder) across three deployments: Non-EAP, EAP-Standalone, and full EAP/RADIUS with FreeRADIUS. At a conservative 256-byte fragment size, total wire bytes range from 7,031 to 11,614 (29 to 48 fragments) and AAA-path handshake time from 56 ms to 440 ms across variants. A first-order energy model derived from the same measurements shows that fragment-induced I/O wait not cryptography dominates the per-handshake energy of an IoT-class endpoint. The results quantify the trade-off between authentication strength, round-trip count, and fragmentation, and give concrete deployment guidance for PQ network-access authentication in e-business IoT settings.

1 Introduction

IoT is increasingly embedded in everyday e-business workflows. Representative business-critical endpoints include *(i)* networked vending machines accepting cashless and QR-code payments; *(ii)* smart point-of-sale (POS) and self-checkout terminals; *(iii)* electronic shelf labels and smart-retail beacons that drive in-store engagement; *(iv)* EV chargers and smart parking meters operated by retail or city operators; *(v)* fleet trackers and cold-chain sensors monitoring logistics conditions; and *(vi)* connected industrial sensors and programmable logic-controller (PLC) gateways feeding predictive-maintenance pipelines on the factory floor. Each of these devices is, in commercial terms, a small business endpoint

*Proceedings of The 2026 IFIP WG 8.4 International Symposium on E-Business Information Systems Evolution (EBISION 2026), Article No. 25, July 8-10, 2026, Seoul, South Korea.
© The copyright of this paper remains with the author(s).
†Corresponding author

that handles payment data, customer behaviour, or operational telemetry, and must therefore be enrolled and authenticated before exchanging data with the back-office. As such endpoints onboard onto enterprise networks most often via Wi-Fi or other non-3GPP access *network access authentication* is the first line of defence that establishes device identity, derives session keys, and gates the device onto production traffic. EAP [1] provides the dominant framework for this task: it is used by IEEE 802.1X, Wi-Fi WPA2/3-Enterprise, and 5G-AKA, and it typically runs over a RADIUS/Diameter backend in which a central Authentication, Authorization and Accounting (AAA) server makes the final authentication decision. Among the many EAP methods, EAP-TLS [2], EAP-AKA [3] and EAP-PSK [4] are the most widely deployed. For constrained IoT devices, however, EAP-TLS is expensive in bytes on the wire and certificate chains [5, 6]. EDHOC [7, 8] is a lightweight authenticated Diffie–Hellman key exchange using Concise Binary Object Representation (CBOR) [9] and CBOR Object Signing and Encryption (COSE) [10], designed for constrained networks and standardized by the IETF LAKE working group [11]. Recent work proposes embedding EDHOC as an EAP method [12, 6], which brings EDHOC's compact flights to mainstream AAA-based network access.

At the same time, Shor's algorithm [13] threatens every elliptic-curve deployment in use today. While a cryptanalytically relevant quantum computer is not yet available, the *harvest-now, decrypt-later* adversary model is already a documented concern for long-lived business data such as payment records and supply-chain telemetry, and motivates beginning the transition before such a machine appears. NIST has standardized ML-KEM (FIPS 203) [14] and ML-DSA (FIPS 204) [15] as first-line post-quantum replacements. Regulators in France [16] and Germany [17] and the IETF PQUIP working group [18] recommend *hybrid* PQ/traditional combinations during the transition period. A post-quantum variant of EDHOC (`draft-papon-lake-pq-edhoc`) is under active specification [19] and defines five authentication modes in its Sections 2, 3.2, 3.3, 3.4 and 3.5; follow-up analyses have evaluated quantum-resistant EDHOC on microcontrollers [20, 21].

What is not yet understood is what happens when these PQ-EDHOC variants are carried by *EAP in a real AAA setting*. EAP defines a minimum MTU of only 1020 octets and mandates a lock-step, fragment-ACK flow for payloads larger than that [1, 22]. ML-KEM ciphertexts and ML-DSA signatures push individual EDHOC messages into the multi-kilobyte range, so each handshake message incurs multiple fragments and additional round-trips end-to-end between the EAP peer, the authenticator, and the AAA server. Prior empirical EDHOC studies [5, 20, 21] evaluated EDHOC peer-to-peer and did not exercise an EAP/RADIUS path; prior EAP-EDHOC work [12, 6] has focused on the classical primitive set.

Our work sits at the intersection of four threads: (i) the EAP framework itself and its canonical methods [1, 2, 3, 4]; (ii) EDHOC and its integration with EAP [7, 8, 11, 12, 6, 5]; (iii) post-quantum and hybrid key exchange, including FIPS 203/204 [14, 15], ETSI TS 103 744 [23], PQXDH [24, 25], PQ Noise [26], hybrid IKEv2 [27], and hybrid TLS 1.3 [28]; and (iv) formal and computational analyses of EDHOC such as [29, 30, 31, 32, 33, 34, 35]. The KEM-based EDHOC authentication Internet-Draft [36] and the signature/KEM PQ-EDHOC draft [19] are the closest antecedents to our proposal, and stronger KEM notions such as those in [37] motivate why KEM-authenticated handshakes deserve careful symbolic treatment. Despite substantial progress in each of these threads, four gaps remain. *First*, the impact of PQ key/ciphertext enlargement on EAP fragmentation under the minimum 1020-octet EAP MTU has not been quantified. *Second*, formal symbolic analyses of the PQ variants of EAP-EDHOC have not been published, and existing ProVerif models target classical EDHOC [30, 33]. *Third*, empirical evaluations of PQ-EDHOC have so far targeted EDHOC peer-to-peer or microcontrollers without a real AAA backend [20, 21]. *Fourth*, the different PQ-EDHOC authentication modes (KEM-only, signature-only, and hybrid) have not been compared uniformly in a deployment with EAP and a real AAA server such as FreeRADIUS.

1.1 Contribution

We address these gaps with the following contributions. Throughout the paper we refer to the five authentication variants of `draft-papon-lake-pq-edhoc` by their draft-section numbers, namely Section 2 [19], Section 3.2 [19], Section 3.3 [19], Section 3.4 [19] and Section 3.5 [19]; this nomenclature is used uniformly in every algorithm, table, and figure of the paper.

- We specify EAP-EDHOC-PQ, a unified EAP method that embeds the five PQ-EDHOC authentication variants of [19] — the implicitly KEM-authenticated three-flight mode (Section 2 [19], Section 3.1); the signature+KEM three-flight mode (Section 3.2 [19], Section 3.2); and three four-flight hybrid variants (Section 3.3 [19], Section 3.4 [19] and Section 3.5 [19], Sections 3.3–3.5) over EAP and RADIUS.
- We provide a ProVerif model of all five variants and prove mutual authentication, session-key secrecy, forward secrecy, and replay resistance under a Dolev–Yao adversary (Section 4).
- We provide an empirical evaluation on a real testbed with Non-EAP, EAP-Standalone, and full EAP/RADIUS (FreeRADIUS) deployments at a conservative 256-byte fragment size, measuring fragmentation, processing time, overhead, and per-operation latency (Section 5).
- We derive concrete deployment guidance: which variant minimizes wire bytes, which minimizes round-trips, and how the AAA path amplifies the cost of four-flight hybrid modes (Section 6).

The remainder of the paper is organized as follows. Section 2 introduces the primitives, the EAP/RADIUS model, and notation. Section 3 specifies the unified EAP-EDHOC-PQC design and its five variants. Section 4 presents the ProVerif analysis. Section 5 reports the empirical evaluation. Section 6 discusses implications and limitations. Section 7 concludes.

2 Preliminaries

This section fixes notation, recalls the cryptographic primitives we use, summarizes EDHOC and EAP, and introduces the threat model. Table 1 lists the abbreviations used throughout the paper.

We use NIST-standardized post-quantum primitives. **ML-KEM** [14] is an IND-CCA2-secure key-encapsulation mechanism with three algorithms $(\mathsf{KeyGen}, \mathsf{KEM.Encaps}, \mathsf{KEM.Decaps})$. **ML-DSA** [15] is a module-lattice digital signature algorithm. Symmetric primitives follow RFC 9528: SHA-256 for hashing, HKDF [38] for key derivation, and AES-CCM (an AEAD) for authenticated encryption.

EDHOC is a three-message (optionally four-message) authenticated key exchange encoded in CBOR [9, 10]. The canonical Signature-Signature (SIG-SIG) method transmits ephemeral Diffie–Hellman contributions and authenticates both parties with digital signatures; the Static-DH modes use long-term DH keys and derive MAC tags instead. A detailed security analysis in the classical setting is provided in [30, 31, 32, 33, 34, 35].

EAP [1] is a lock-step request/response protocol operating over a lower layer (e.g., IEEE 802.1X, PPP, or IKEv2). For EAP-over-RADIUS, an *authenticator* (typically a gateway or access point) relays EAP packets between the peer and the AAA server inside RADIUS attributes. Two properties are central to our evaluation. *(a)* EAP implementations may assume an MTU of only 1020 octets from the lower layer, and EAP itself does not provide fragmentation; an EAP method whose messages exceed the MTU must implement fragmentation with explicit ACKs, at the cost of extra round-trips [1, 22]. *(b)* RADIUS packets are capped at 4096 octets and carry EAP payloads in multiple `EAP-Message` AVPs of 253 bytes each; long EAP messages therefore cross both the EAP fragmentation boundary and the RADIUS attribute boundary.

We consider a Dolev–Yao adversary [39] controlling the network between the peer, the authenticator, and the AAA server. The adversary may inject, drop, reorder, replay, or modify any message

Table 1: Abbreviations and notation used throughout the paper (including all symbols of Algorithms 1–5 in Section 3).

Symbol	Description	Symbol	Description
Protocols and frameworks			
EAP	Extensible Authentication Protocol (RFC 3748)	EDHOC	Ephemeral Diffie–Hellman over COSE (RFC 9528)
AAA	Authentication, Authorization, Accounting (RADIUS/Diameter)	NAS	Network Access Server (relays EAP between peer and AAA)
RADIUS	Remote Authentication Dial-In User Service (RFC 2865)	CBOR	Concise Binary Object Representation (RFC 8949)
COSE	CBOR Object Signing and Encryption (RFC 9052)	MTU	Maximum Transmission Unit
LAKE	IETF working group for Lightweight AKE		
Cryptographic primitives			
KEM	Key Encapsulation Mechanism	ML-KEM	Module-Lattice KEM (FIPS 203)
ML-DSA	Module-Lattice Digital Signature Algorithm (FIPS 204)	DS	Digital Signature scheme (instantiated by ML-DSA)
AEAD	Authenticated Encryption with Associated Data	HKDF	HMAC-based Extract-and-Expand KDF (RFC 5869)
KDF	Key Derivation Function	HMAC	Hashed Message Authentication Code
MAC	Message Authentication Code	SHA-256	Secure Hash Algorithm, 256-bit output
$H(\cdot)$	cryptographic hash (SHA-256 in our instantiation)	$\mathsf{KeyGen}, \mathsf{KEM.Encaps}, \mathsf{KEM.Decaps}$	KEM key generation, encapsulation, decapsulation
DS.Sign, **DS.Verify**	digital-signature signing / verification	**AEAD.Enc**, **AEAD.Dec**	AEAD encryption / decryption
Parties, identifiers and credentials			
I, R	EDHOC Initiator, Responder	C_I, C_R	EDHOC connection identifiers chosen by I, R
ID_CRED_I, ID_CRED_R	credential identifier of I, R	METHOD, SUITES_I	EDHOC method byte and offered cipher-suite list
EAD_i	External Authorization Data field of message i	MSK, EMSK	Master Session Key, Extended MSK exported to EAP
Per-handshake key material			
sk_X, pk_X	long-term secret/public key of party X	kem.sk_X, kem.pk_X	long-term KEM key pair of party X
sign.sk_X, sign.pk_X	long-term signing key pair of party X	$\text{kem.sk}_{\text{eph}}$, $\text{kem.pk}_{\text{eph}}$	ephemeral KEM key pair generated by I
$\text{kem.ct}_{\text{eph}}$	KEM ciphertext encapsulating to $\text{kem.pk}_{\text{eph}}$	kem.ct_R, kem.ct_I	KEM ciphertexts encapsulating to R's, I's long-term key
ss_{eph}, ss_R, ss_I	shared secrets from the three KEM operations	TH_i	EDHOC transcript hash after message i
PRK_{2e}	PRK from ss_{eph}	PRK_{3e2m}	PRK after mixing ss_R
PRK_{4e3m}	PRK after mixing ss_I (four-flight modes)	PRK_{out}	exporter PRK feeding MSK/EMSK derivation
$SALT_{3e2m}$, $SALT_{4e3m}$	salts used in the corresponding HKDF-Extract	$KEYSTREAM_2$	XOR keystream protecting $PLAINTEXT_2$
K_3, IV_3, K_4, IV_4	AEAD key/IV pairs for messages 3 and 4	MAC_i	authentication tag carried in (or covering) flight i
$SIGNATURE_i$	ML-DSA signature attached to flight i	$PLAINTEXT_i$, $CIPHERTEXT_i$	plaintext / ciphertext payload of message i
$\|$	byte-string concatenation		

and may corrupt long-term keys of peers other than the target session for forward secrecy arguments. Cryptographic primitives are idealized as perfect symbolic functions, consistent with ProVerif's applied-pi-calculus model and standard practice [40, 41, 42].

3 Proposed Design: EAP-EDHOC-PQ

The proposed **EAP-EDHOC-PQ** method carries PQ-EDHOC flights between the EAP peer (the EDHOC Initiator I, typically an IoT device), the authenticator, and the AAA server (the EDHOC Responder R). The unified design supports five authentication variants Section 2 [19], Section 3.2 [19], Section 3.3 [19], Section 3.4 [19], and Section 3.5 [19] that trade off the authentication mechanism (KEM, signature, or hybrid), the number of flights (three or four), and the timing of responder authentication. Figure 1 summarizes the complete protocol flow. Sections 3.1–3.5 describe the variants individually. Common header (EAP Identity and EDHOC-Start). All variants begin with the standard EAP Identity exchange: the authenticator relays the peer's EAP-Response/Identity to the AAA server, which answers with a RADIUS Access-Challenge carrying EAP-Request/EDHOC-Start. From this point on the five variants diverge. While EDHOC defines message_4 as optional, the EAP-EDHOC draft mandates the use of message_4 as a protected success indication. In this work, Sections 2 and 3.2 allow an optimized three-flight operation by omitting message_4, whereas Sections 3.3–3.5 retain the four-message exchange for full EAP-EDHOC compatibility.

3.1 Variant 1 Section 2 [19]

Section 2 [19] authenticates the Initiator explicitly with an ML-DSA signature and the Responder *implicitly* through ML-KEM, following the philosophy of [36]. Before the handshake the Initiator knows the Responder's long-term KEM public key $kem.pk_R$. Message 1 carries an ephemeral KEM public key $kem.pk_{eph}$ together with the static-to-Responder ciphertext $kem.ct_R \leftarrow \mathsf{KEM.Encaps}(kem.pk_R)$, which delivers ss_R to the Responder. The Responder then encapsulates to the Initiator's ephemeral KEM key, producing $(ss_{eph}, kem.ct_{eph})$, and mixes both shared secrets into $PRK_{3e2m} = \text{HKDF-Extract}(SALT_{3e2m}, ss_R)$. The Responder is authenticated by its ability to decapsulate ss_R, which is the MAC key that Message 3 verifies. Message 4 is optional (key confirmation only). This variant is the smallest on the wire and uses a single KEM decapsulation at R per handshake.

3.2 Variant 2 Section 3.2 [19]

Section 3.2 [19] provides explicit Responder authentication via ML-DSA in Message 2 while still achieving a three-flight main handshake. Message 1 carries only $kem.pk_{eph}$; the Responder encapsulates to it and signs $PLAINTEXT_2$ with its ML-DSA key. The Initiator encapsulates to $kem.pk_R$ in Message 3, delivering ss_R to the Responder and binding the handshake to the Responder's long-term KEM identity. Compared with Section 2 [19], Section 3.2 [19] replaces the implicit KEM authentication of R with an explicit signature in Message 2 and therefore pays a larger Message 2 on the wire.

3.3 Variant 3 Section 3.3 [19]

Section 3.3 [19] is the *mirror* of Section 3.2 [19] with respect to the direction of the static KEM. Authentication of I uses both an ML-DSA signature and KEM confirmation *to the Initiator*, so the Responder must encapsulate to $kem.pk_I$ after learning ID_CRED_I in Message 3. This forces Message 4 to become *mandatory*: it carries $kem.ct_I$ and the encrypted confirmation. Section 3.3 [19] therefore yields a four-flight handshake, with the natural cost that the AAA path adds one more RADIUS round-trip.

3.4 Variant 4 Section 3.4 [19]

Section 3.4 [19] implements KEM+KEM authentication on both sides but defers Responder authentication: the Responder's MAC tag is not sent in Message 2 but carried inside Message 4, after R has

encapsulated to $kem.pk_I$. The design yields three shared secrets $\{ss_{eph}, ss_R, ss_I\}$ and the strongest key-contributiveness among our variants, at the cost of one additional KEM operation on each side.

3.5 Variant 5 Section 3.5 [19]

Section 3.5 [19] combines the best of the hybrid options: explicit ML-DSA authentication of R in Message 2 *and* KEM-based authentication of I via Message 4. It is the "full hybrid" variant and provides the strongest authentication guarantees in our family at the cost of the highest wire size and the most cryptographic operations.

Every EDHOC message is carried inside an EAP-Request/EAP-Response of type `EAP-EDHOC-PQC`. When the EAP payload exceeds the negotiated fragment size, EAP-EDHOC-PQC uses an explicit flag-based fragmentation with `M`-bit ("more fragments"), mirroring EAP-TLS and the classical EAP-EDHOC draft [12, 6]. Each fragment carries a 4-byte EAP header; the sender waits for an ACK fragment (an EAP-Response of type EAP-EDHOC-PQC with empty data) before sending the next fragment, introducing one lock-step round-trip per fragment pair. The original PQ-EDHOC Internet-Draft [19] groups its five variants under two umbrella modes (SIG-SIG and MAC-MAC) and develops the SIG-SIG mode as a single Section 3.2 example, without any consideration of EAP transport. Closely related work either keeps EDHOC peer-to-peer over UDP/CoAP [20, 21, 5] or embeds *classical* EDHOC into EAP [12, 6]. No prior design carries all five PQ-EDHOC variants over a real EAP/RADIUS path with an off-the-shelf AAA server, and prior comparisons therefore cannot speak to the cost of EAP fragmentation, the RADIUS attribute boundary, or the lock-step ACK round-trip. Our unified specification fills this gap. Section 2 [19] is the only mode that retains the three-flight structure *and* drops one signature relative to SIG-SIG, while Section 3.4 [19] is, to the best of our knowledge, the first EAP-friendly KEM+KEM hybrid with deferred Responder authentication suitable for fragmentation under the EAP MTU. Two dimensions distinguish our design from the alternatives. *First, EAP transport vs. peer-to-peer.* A peer-to-peer PQ-EDHOC handshake travels over a single UDP/CoAP [7] association between I and R, with no fragmentation contract beyond what the lower layer offers; in contrast, EAP imposes a lock-step request/response state machine, an MTU as low as 1020 octets [1, 22], and a fragment-ACK exchange that adds one round-trip per fragment pair. The same multi-kilobyte ML-DSA signature therefore costs proportionally more wall-clock time on EAP than in a peer-to-peer setting a cost that is invisible from the draft's own description. *Second, AAA backend vs. standalone.* An EAP-Standalone deployment terminates EAP at the authenticator, while a full AAA deployment relays each EAP message inside RADIUS `EAP-Message` attributes (capped at 253 bytes each [43]) over a separate Network Access Server $\leftrightarrow$ AAA hop. The RADIUS hop adds latency but also amortizes some I/O via FreeRADIUS's batched packetization, and the joint behaviour cannot be inferred from the peer-to-peer numbers alone.

Algorithm 1 Step 1 (Initiator) Prepare M_1

Input: Longterm PQ key pair (pk_I, sk_I); Responder's kem.pk_R (Section 2 only)
Output: M_1

[All Sections]
$(\text{kem.sk}_{\text{eph}}, \text{kem.pk}_{\text{eph}}) \leftarrow \textbf{KEM.KeyGen}()$
[Section 2 only]
$(\text{ss}_R, \text{kem.ct}_R) \leftarrow \textbf{KEM.Encaps}(\text{kem.pk}_R)$
$M_1 \leftarrow (\text{METHOD}, \text{SUITES}_I, \text{kem.pk}_{\text{eph}}, \text{kem.ct}_R, C_I, \text{EAD}_1)$
[Section 3.2, 3.3, 3.4, 3.5]
$M_1 \leftarrow (\text{METHOD}, \text{SUITES}_I, \text{kem.pk}_{\text{eph}}, C_I, \text{EAD}_1)$
return M_1

Algorithm 1 describes the generation of M_1. The Initiator generates an ephemeral KEM key pair and includes the public key in the message. In Section 2, assuming an authenticated copy of the Responder's KEM public key, it additionally produces kem.ct_R through KEM encapsulation to enable implicit Responder authentication. The resulting M_1 contains the protocol parameters, connection identifier, optional authorization data, and, when applicable, kem.ct_R.



Figure 1: Unified EAP-EDHOC-PQC over EAP/RADIUS, five variants

Algorithm 2 Step 2 (Responder) Process M_1 and Prepare M_2

Input: M_1; Responder's long-term keys (KEM and/or signing, per section)
Output: M_2

[All Sections]
$(\text{ss}_{\text{eph}}, \text{kem.ct}_{\text{eph}}) \leftarrow \textbf{KEM.Encaps}(\text{kem.pk}_{\text{eph}})$
$TH_2 \leftarrow H(\text{kem.ct}_{\text{eph}}, H(M_1))$
$PRK_{2e} \leftarrow \textbf{EDHOC-Extract}(TH_2, \text{ss}_{\text{eph}})$
$KEYSTREAM_2 \leftarrow \textbf{EDHOC-KDF}(PRK_{2e}, 0, TH_2)$
[Section 2] ▷ implicit R-auth via ss_R from M_1
$\text{ss}_R \leftarrow \textbf{KEM.Decaps}(\text{kem.sk}_R, \text{kem.ct}_R)$
$SALT_{3e2m} \leftarrow \textbf{EDHOC-KDF}(PRK_{2e}, 1, TH_2)$
$PRK_{3e2m} \leftarrow \textbf{EDHOC-Extract}(SALT_{3e2m}, \text{ss}_R)$
$MAC_2 \leftarrow \textbf{KDF}(PRK_{3e2m}, 2, (C_R, ID_CRED_R, TH_2, \text{EAD}_2))$
$PLAINTEXT_2 \leftarrow (C_R, ID_CRED_R, MAC_2, \text{EAD}_2)$
[Section 3.2, 3.5] ▷ explicit R-auth via signature over $PLAINTEXT_2$
$PLAINTEXT_2 \leftarrow (C_R, ID_CRED_R, TH_2, \text{EAD}_2)$
$SIGNATURE_2 \leftarrow \textbf{DS.Sign}(\text{sign.sk}_R, PLAINTEXT_2)$
$PLAINTEXT_2 \leftarrow (PLAINTEXT_2 \parallel SIGNATURE_2)$
[Section 3.3] ▷ MAC_2 + signature over MAC_2, MAC_2 derived from PRK_{2e}
$MAC_2 \leftarrow \textbf{KDF}(PRK_{2e}, 2, (C_R, ID_CRED_R, TH_2, \text{EAD}_2))$
$SIGNATURE_2 \leftarrow \textbf{DS.Sign}(\text{sign.sk}_R, (C_R, ID_CRED_R, TH_2, \text{EAD}_2, MAC_2))$
$PLAINTEXT_2 \leftarrow (C_R, ID_CRED_R, SIGNATURE_2, \text{EAD}_2)$
[Section 3.4] ▷ MAC_2 *deferred* to M_4; M_2 carries no auth material
$PLAINTEXT_2 \leftarrow (C_R, ID_CRED_R, TH_2, \text{EAD}_2)$
[All Sections]
$CIPHERTEXT_2 \leftarrow PLAINTEXT_2 \oplus KEYSTREAM_2$
$M_2 \leftarrow (\text{kem.ct}_{\text{eph}}, CIPHERTEXT_2)$
return M_2

As we can see at Figure 1 unified EAP-EDHOC-PQC over EAP/RADIUS, comprising five variants:

- Section 2 [19]: KEM-only implicit Responder authentication.
- Section 3.2 [19]: SIG+KEM with three message flights.
- Section 3.3 [19]: SIG+KEM with four message flights and KEM authentication to *I*.
- Section 3.4 [19]: KEM+KEM with deferred Responder authentication.
- Section 3.5 [19]: KEM+KEM+SIG authentication.

Algorithm 2 describes the Responder-side processing of M_1 and the generation of M_2. The Responder first encapsulates to the Initiator's ephemeral KEM public key to obtain the shared secret ss_{eph}, from which TH_2, PRK_{2e}, and the encryption keystream are derived. The authentication mechanism then depends on the selected variant. Section 2 performs implicit authentication through a long-term KEM decapsulation and the derivation of MAC_2. Sections 3.2 and 3.5 provide explicit authentication using a digital signature over the protected protocol data. Section 3.3 combines a transcript-bound MAC_2 with a signature, while Section 3.4 defers Responder authentication to M_4 and therefore includes no authentication material in M_2. Finally, the plaintext is encrypted using the derived keystream, and M_2 is formed by combining the ephemeral KEM ciphertext with the protected payload.

Algorithm 3 Step 3 (Initiator) Process M_2 and Prepare M_3

Input: M_2
Output: M_3

[All Sections]
$\text{ss}_{\text{eph}} \leftarrow \textbf{KEM.Decaps}(\text{kem.sk}_{\text{eph}}, \text{kem.ct}_{\text{eph}})$
Compute $TH_2, PRK_{2e}, KEYSTREAM_2$ and recover $PLAINTEXT_2$
$TH_3 \leftarrow H(TH_2, PLAINTEXT_2, ID_CRED_R)$
[Section 2] ▷ verify $MAC_2 \Rightarrow$ implicit R-auth
$PRK_{3e2m} \leftarrow \textbf{EDHOC-Extract}(SALT_{3e2m}, \text{ss}_R)$
Verify $MAC_2 \stackrel{?}{=} \textbf{KDF}(PRK_{3e2m}, 2, \cdot)$
$(K_3, IV_3) \leftarrow \textbf{EDHOC-KDF}(PRK_{3e2m}, 3/4, TH_3)$
$MAC_3 \leftarrow \textbf{KDF}(PRK_{3e2m}, 6, (ID_CRED_I, TH_3, \text{EAD}_3))$
$SIGNATURE_3 \leftarrow \textbf{DS.Sign}(\text{sign.sk}_I, (ID_CRED_I, TH_3, \text{EAD}_3, MAC_3))$
$PLAINTEXT_3 \leftarrow (ID_CRED_I, SIGNATURE_3, \text{EAD}_3)$
$M_3 \leftarrow \textbf{AEAD.Enc}(K_3, IV_3, PLAINTEXT_3)$ ▷ no kem.ct_R sent in M_1
[Section 3.2, 3.5] ▷ explicit R-auth via signature verification; new kem.ct_R
Retrieve $\text{kem.pk}_R, \text{sign.pk}_R$; verify $\textbf{DS.Verify}(\text{sign.pk}_R, PLAINTEXT_2, SIGNATURE_2)$
$(\text{ss}_R, \text{kem.ct}_R) \leftarrow \textbf{KEM.Encaps}(\text{kem.pk}_R)$
$PRK_{3e2m} \leftarrow \textbf{EDHOC-Extract}(SALT_{3e2m}, \text{ss}_R)$
$(K_3, IV_3) \leftarrow \textbf{EDHOC-KDF}(PRK_{3e2m}, 3/4, TH_3)$
$MAC_3 \leftarrow \textbf{KDF}(PRK_{3e2m}, 6, (ID_CRED_I, TH_3, \text{EAD}_3))$
$SIGNATURE_3 \leftarrow \textbf{DS.Sign}(\text{sign.sk}_I, (ID_CRED_I, TH_3, \text{EAD}_3, MAC_3))$
$CIPHERTEXT_3 \leftarrow \textbf{AEAD.Enc}(K_3, IV_3, (ID_CRED_I, SIGNATURE_3, \text{EAD}_3))$
$M_3 \leftarrow (\text{kem.ct}_R, CIPHERTEXT_3)$
[Section 3.3] ▷ sign $PLAINTEXT_3$ directly; no MAC_3; no kem.ct_R
$MAC_2 \leftarrow \textbf{KDF}(PRK_{2e}, 2, \cdot)$; verify $\textbf{DS.Verify}(\text{sign.pk}_R, \cdot, SIGNATURE_2)$
$(K_3, IV_3) \leftarrow \textbf{EDHOC-KDF}(PRK_{2e}, 3/4, TH_3)$
$PLAINTEXT_3 \leftarrow (ID_CRED_I, TH_3, \text{EAD}_3)$
$SIGNATURE_3 \leftarrow \textbf{DS.Sign}(\text{sign.sk}_I, PLAINTEXT_3)$
$M_3 \leftarrow \textbf{AEAD.Enc}(K_3, IV_3, (PLAINTEXT_3 \parallel SIGNATURE_3))$
[Section 3.4] ▷ no SIG_2 verify yet R-auth deferred to M_4; new kem.ct_R
$(\text{ss}_R, \text{kem.ct}_R) \leftarrow \textbf{KEM.Encaps}(\text{kem.pk}_R)$
$PRK_{3e2m} \leftarrow \textbf{EDHOC-Extract}(SALT_{3e2m}, \text{ss}_R)$
$(K_3, IV_3) \leftarrow \textbf{EDHOC-KDF}(PRK_{3e2m}, 3/4, TH_3)$
$SIGNATURE_3 \leftarrow \textbf{DS.Sign}(\text{sign.sk}_I, (ID_CRED_I, TH_3, \text{EAD}_3))$
$CIPHERTEXT_3 \leftarrow \textbf{AEAD.Enc}(K_3, IV_3, (ID_CRED_I, TH_3, \text{EAD}_3, SIGNATURE_3))$
$M_3 \leftarrow (\text{kem.ct}_R, CIPHERTEXT_3)$
return M_3

Algorithm 3 describes the Initiator-side processing of M_2 and the generation of M_3. After decapsulating the ephemeral KEM ciphertext, the Initiator derives TH_2, PRK_{2e}, and the associated protection keys before recovering $PLAINTEXT_2$. The subsequent authentication and key-derivation steps depend on the selected variant. In Section 2, implicit Responder authentication is achieved through verification of MAC_2 using the previously established long-term KEM secret. Sections 3.2 and 3.5 perform explicit Responder authentication through signature verification and establish an additional KEM shared secret for mutual authentication. Section 3.3 verifies the Responder's signature and derives subsequent keys directly from the ephemeral exchange, while Section 3.4 postpones Responder authentication until M_4 and

therefore proceeds without verifying any authentication evidence contained in M_2. Finally, the Initiator constructs and protects its authentication payload, producing M_3 for transmission to the Responder.

Algorithm 4 Step 4 (Responder) Process M_3 and Prepare M_4

Input: M_3

Output: M_4 (optional for Sections 2 and 3.2; mandatory for Sections 3.3, 3.4 and 3.5), PRK_{out}

[All Sections]

Derive keys, decrypt $CIPHERTEXT_3$, and recover $PLAINTEXT_3$

Verify MAC_3 and/or $SIGNATURE_3$ per section (details below)

[Section 2, 3.2] ▷ M_4 optional; carries only EAD_4

Verify $\textbf{DS.Verify}(\text{sign.pk}_I, (ID_CRED_I, TH_3, \text{EAD}_3, MAC_3), SIGNATURE_3)$

$TH_4 \leftarrow H(TH_3, PLAINTEXT_3, ID_CRED_I)$

$(K_4, IV_4) \leftarrow \textbf{EDHOC-KDF}(PRK_{3e2m}, 8/9, TH_4)$

$M_4 \leftarrow \textbf{AEAD.Enc}(K_4, IV_4, \text{EAD}_4)$ *(if requested)*

$PRK_{\text{out}} \leftarrow \textbf{KDF}(PRK_{3e2m}, 7, TH_4)$

[Section 3.3] ▷ M_4 *mandatory*; carries kem.ct_I; $SALT_{4e3m}$ from PRK_{2e}

Verify $\textbf{DS.Verify}(\text{sign.pk}_I, PLAINTEXT_3, SIGNATURE_3)$

$(\text{ss}_I, \text{kem.ct}_I) \leftarrow \textbf{KEM.Encaps}(\text{kem.pk}_I)$

$TH_4 \leftarrow H(TH_3, PLAINTEXT_3, ID_CRED_I)$

$SALT_{4e3m} \leftarrow \textbf{KDF}(PRK_{2e}, 5, TH_4)$

$PRK_{4e3m} \leftarrow \textbf{EDHOC-Extract}(SALT_{4e3m}, \text{ss}_I)$

$(K_4, IV_4) \leftarrow \textbf{EDHOC-KDF}(PRK_{4e3m}, 8/9, TH_4)$

$M_4 \leftarrow (\text{kem.ct}_I, \textbf{AEAD.Enc}(K_4, IV_4, \text{EAD}_4))$

$PRK_{\text{out}} \leftarrow \textbf{KDF}(PRK_{4e3m}, 7, TH_4)$

[Section 3.4] ▷ M_4 mandatory; carries *deferred* MAC_2

$\text{ss}_R \leftarrow \textbf{KEM.Decaps}(\text{kem.sk}_R, \text{kem.ct}_R)$

Verify $\textbf{DS.Verify}(\text{sign.pk}_I, (ID_CRED_I, TH_3, \text{EAD}_3), SIGNATURE_3)$

$(\text{ss}_I, \text{kem.ct}_I) \leftarrow \textbf{KEM.Encaps}(\text{kem.pk}_I)$

$TH_4 \leftarrow H(TH_3, PLAINTEXT_3, ID_CRED_I)$

$SALT_{4e3m} \leftarrow \textbf{KDF}(PRK_{3e2m}, 5, TH_4)$

$PRK_{4e3m} \leftarrow \textbf{EDHOC-Extract}(SALT_{4e3m}, \text{ss}_I)$

$(K_4, IV_4) \leftarrow \textbf{EDHOC-KDF}(PRK_{4e3m}, 8/9, TH_4)$

$MAC_2 \leftarrow \textbf{KDF}(PRK_{4e3m}, 2, (C_R, ID_CRED_R, TH_4, \text{EAD}_4))$

$M_4 \leftarrow (\text{kem.ct}_I, \textbf{AEAD.Enc}(K_4, IV_4, (\text{EAD}_4, MAC_2)))$

$PRK_{\text{out}} \leftarrow \textbf{KDF}(PRK_{4e3m}, 7, TH_4)$

[Section 3.5] ▷ M_4 mandatory; carries only EAD_4; SIG_3 over $PLAINTEXT_3$

$\text{ss}_R \leftarrow \textbf{KEM.Decaps}(\text{kem.sk}_R, \text{kem.ct}_R)$

Verify $\textbf{DS.Verify}(\text{sign.pk}_I, PLAINTEXT_3, SIGNATURE_3)$

$(\text{ss}_I, \text{kem.ct}_I) \leftarrow \textbf{KEM.Encaps}(\text{kem.pk}_I)$

$TH_4 \leftarrow H(TH_3, PLAINTEXT_3, ID_CRED_I)$

$SALT_{4e3m} \leftarrow \textbf{KDF}(PRK_{3e2m}, 5, TH_4)$

$PRK_{4e3m} \leftarrow \textbf{EDHOC-Extract}(SALT_{4e3m}, \text{ss}_I)$

$(K_4, IV_4) \leftarrow \textbf{EDHOC-KDF}(PRK_{4e3m}, 8/9, TH_4)$

$M_4 \leftarrow (\text{kem.ct}_I, \textbf{AEAD.Enc}(K_4, IV_4, \text{EAD}_4))$

$PRK_{\text{out}} \leftarrow \textbf{KDF}(PRK_{4e3m}, 7, TH_4)$

return M_4, PRK_{out}

Algorithm 4 describes the Responder-side processing of M_3 and the generation of the final protocol

message M_4, when required. After deriving the appropriate keys and decrypting $CIPHERTEXT_3$, the Responder verifies the Initiator's authentication evidence, either through MAC_3, $SIGNATURE_3$, or both, depending on the selected variant. For Sections 2 and 3.2, authentication is completed after verification and M_4 is optional. In contrast, Sections 3.3, 3.4, and 3.5 require an additional KEM exchange and a mandatory M_4 to complete key confirmation and derive the final session secret. Section 3.4 further carries the deferred MAC_2 in M_4, postponing Responder authentication until the last protocol message. Upon successful completion, the Responder derives PRK_{out}, which serves as the exported session secret for subsequent EAP and application-layer key derivation.

Algorithm 5 Step 5 (Initiator) Process M_4 (optional for Sections 2 and 3.2; mandatory otherwise)

Input: M_4
Output: PRK_{out}

[Section 2, 3.2] ▷ decrypt EAD_4 only
$(K_4, IV_4) \leftarrow \textbf{EDHOC-KDF}(PRK_{3e2m}, 8/9, TH_4)$
$\text{EAD}_4 \leftarrow \textbf{AEAD.Dec}(K_4, IV_4, CIPHERTEXT_4)$
$PRK_{\text{out}} \leftarrow \textbf{KDF}(PRK_{3e2m}, 7, TH_4)$
[Section 3.3, 3.5] ▷ decaps kem.ct_I; $SALT_{4e3m}$ source differs per section
$\text{ss}_I \leftarrow \textbf{KEM.Decaps}(\text{kem.sk}_I, \text{kem.ct}_I)$
$SALT_{4e3m} \leftarrow \textbf{KDF}(PRK_{2e \text{ or } 3e2m}, 5, TH_4)$ ▷ PRK_{2e} for Section 3.3; PRK_{3e2m} for Section 3.5
$PRK_{4e3m} \leftarrow \textbf{EDHOC-Extract}(SALT_{4e3m}, \text{ss}_I)$
$(K_4, IV_4) \leftarrow \textbf{EDHOC-KDF}(PRK_{4e3m}, 8/9, TH_4)$
$\text{EAD}_4 \leftarrow \textbf{AEAD.Dec}(K_4, IV_4, CIPHERTEXT_4)$
$PRK_{\text{out}} \leftarrow \textbf{KDF}(PRK_{4e3m}, 7, TH_4)$
[Section 3.4] ▷ verify *deferred* MAC_2 here ⇒ explicit R-auth
$\text{ss}_I \leftarrow \textbf{KEM.Decaps}(\text{kem.sk}_I, \text{kem.ct}_I)$
$SALT_{4e3m} \leftarrow \textbf{KDF}(PRK_{3e2m}, 5, TH_4)$
$PRK_{4e3m} \leftarrow \textbf{EDHOC-Extract}(SALT_{4e3m}, \text{ss}_I)$
$(K_4, IV_4) \leftarrow \textbf{EDHOC-KDF}(PRK_{4e3m}, 8/9, TH_4)$
$(\text{EAD}_4, MAC_2) \leftarrow \textbf{AEAD.Dec}(K_4, IV_4, CIPHERTEXT_4)$
Verify $MAC_2 \stackrel{?}{=} \textbf{KDF}(PRK_{4e3m}, 2, (C_R, ID_CRED_R, TH_4, \text{EAD}_4))$
$PRK_{\text{out}} \leftarrow \textbf{KDF}(PRK_{4e3m}, 7, TH_4)$
return PRK_{out}

Algorithm 5 describes the final processing stage performed by the Initiator after receiving M_4. For Sections 2 and 3.2, the procedure is limited to decrypting the optional protected $\text{EAD} * 4$ payload and deriving the exported secret $PRK * \text{out}$. In Sections 3.3 and 3.5, the Initiator first decapsulates the additional KEM ciphertext carried in M_4, derives PRK_{4e3m}, and then decrypts the protected payload before generating the final session secret. Section 3.4 follows the same key-derivation process but additionally verifies the deferred MAC_2 contained in M_4, thereby completing explicit Responder authentication. Upon successful verification, the Initiator derives PRK_{out}, concluding the EDHOC-PQ handshake and providing the shared secret used for subsequent EAP and application-layer key derivation.

4 Security Analysis

We model each of the five EAP-EDHOC-PQC variants in the applied pi calculus [40, 41] and verify the security requirements summarized in Table 2 under the Dolev–Yao adversary [39, 42]. The complete ProVerif source code for all five variants `section2.pv`, `section32.pv`, `section33.pv`, `section34.pv`

Table 2: Security requirements and the corresponding ProVerif verification queries used for every EAP-EDHOC-PQC variant. The same Q1–Q10 query set is instantiated in each per-variant ProVerif file (`section2.pv`, `section32.pv`, `section33.pv`, `section34.pv`, `section35.pv`). Q1–Q7 are active by default; Q8–Q10 are enabled by uncommenting the corresponding inline hook (`LeakFS`, `[Q9 AEAD LEAK]`, `[Q10 PQ LEAK]`).

Security Requirement	**Verification Query**
Agreement on M_1	**Q1:** `inj-event(E_STEP1_I_to_R(m)) ==> inj-event(S_STEP1_I_to_R(m))`
Mutual Authentication	**Q2:** `inj-event(E_STEP2_R_to_I(mac,msg,th)) ==> inj-event(S_STEP2_R_to_I(mac,msg,th))` **Q3:** `inj-event(E_STEP3_I_to_R(mac,msg,th)) ==> inj-event(S_STEP3_I_to_R(mac,msg,th))`
Identity Concealment	**Q4:** `query attacker(I)` **Q5:** `query attacker(R)`
Confidentiality	**Q6:** `query attacker(app_data1)` **Q7:** `query attacker(app_data2)`
Secure Key Exchange	**Q2** $\wedge$ **Q3** $\wedge$ **Q6** $\wedge$ **Q7** i.e., agreement on M_2/M_3 *and* secrecy of *app_data1*, *app_data2*.
Forward Secrecy (FS)	**Q8:** `phase 1; out(c,sk_I); out(c,sk_R); query attacker(app_data1) && attacker(app_data2)`
AEAD Key/IV Reuse Resistance	**Q9:** `phase 1; out(c,EK2); out(c,EK3); query attacker(app_data1) && attacker(app_data2)`
Quantum Resilience	**Q10:** `phase 1; out(c,r_R); out(c,r_eph); query attacker(app_data1) && attacker(app_data2)`

and `section35.pv`, together with the runner script `run_all.sh` is available at https://github.com/adipanca/proverif-ebision and is referenced by file name throughout this section. ML-KEM is modelled as an IND-CCA2 KEM with constructor pair *kem_encap*/*kem_decap*, ML-DSA as an EUF-CMA signature scheme with *sign*/*sverify* constructors, HKDF and SHA-256 as one-way functions, and AEAD as an INT-CTXT primitive with matching *aead_enc*/*aead_dec* constructors. Three channels are used: c_{IN} models the EAP/EDHOC channel between the peer and the authenticator, c_{NA} models the RADIUS hop between the authenticator and the AAA server, and c_{Pub} is a public sink used to expose secrets in the leakage scenarios for forward secrecy, AEAD-key reuse, and quantum resilience.

Following standard practice for replicated EAP/AAA models [30, 33], we use a many-clients/one-server abstraction: the Initiator and the AAA server are replicated, while a single legitimate Responder runs in parallel. This abstraction faithfully captures the EAP threat model in which one NAS/AAA pair serves arbitrarily many (possibly compromised) supplicants, while keeping ProVerif clause saturation tractable in the presence of long-term KEM decapsulation on every M_1 in Section 2 [19]. Table 2 lists the security requirements we verify together with the concrete ProVerif queries that operationalize them. The query set comprises three correspondence queries (Q1–Q3) expressing injective agreement on each EDHOC flight, four secrecy queries (Q4–Q7) covering identity concealment of *I*/*R* and confidentiality of two application-data slots, and three leakage-based queries (Q8–Q10) for forward secrecy, AEAD-key/IV reuse, and quantum resilience respectively. The events S_STEP_i and E_STEP_i are emitted at the *send* and *end* of each EDHOC flight, so that an injective correspondence $\mathsf{inj\text{-}event}(E_STEP_i) \Rightarrow \mathsf{inj\text{-}event}(S_STEP_i)$ proves that every accepting party can be matched to a unique honest sender of that flight.

4.1 Verification Results

We executed every variant under ProVerif 2.05 with the same query set and the option `set traceBacktracking = false` for performance. Table 3 reports the outcome of each query for each variant, using the truth values returned by ProVerif. The notation "T" denotes that the property is verified (secrecy queries succeed as not attacker(·) being true; correspondence queries succeed as the injective implication being true); "F⋆" denotes an *intended* failure that is consistent with the protocol design; "○" denotes a query enabled by uncommenting an inline leakage hook in the corresponding `.pv` file (Q8 toggles `LeakFS`, Q9 toggles `[Q9 AEAD LEAK]`, Q10 toggles `[Q10 PQ LEAK]`).

Table 3: ProVerif verification results per variant

Query	Requirement	Section. 2	Section. 3.2	Section. 3.3	Section. 3.4	Section. 3.5
Q1	Agreement on M_1	F⋆	F⋆	F⋆	F⋆	F⋆
Q2	Agreement on M_2	T	T	T	F⋆	T
Q3	Agreement on M_3	T	T	T	T	T
Q4	Identity concealment of I	T	T	T	T	T
Q5	Identity concealment of R	T	T	T	T	T
Q6	Confidentiality of *app_data1*	T	T	T	T	T
Q7	Confidentiality of *app_data2*	T	T	T	T	T
Q8	Forward secrecy	T	T	T	T	T
Q9	AEAD-key/IV reuse resistance	T	T	T	T	T
Q10	Quantum resilience	T	T	T	T	T

T = property verified; F⋆ = intended failure (see discussion below); Q8,Q9,Q10 = property verified after enabling the corresponding inline leakage hook in the `.pv` file.

4.2 Discussion of the Intended Failures

Two query failures appear in Table 3 and both are intentional and consistent with the protocol specification.

Q1 fails for every variant (M_1 agreement). RFC 9528 §4.5 specifies that M_1 is sent in cleartext: it carries the method, cipher suites, ephemeral KEM public key, optional $kem.ct_R$ in Section 2 [19], the connection identifier C_I, and EAD_1, but no authentication tag. A Dolev–Yao adversary can therefore replay or forge a syntactically valid M_1, which is exactly what ProVerif detects. Q1 is included only as a *sanity check* on the model: a verification that returned `true` would indicate an unintended (and incorrect) authentication shortcut on M_1. Crucially, the lack of authentication on M_1 does not propagate into mutual authentication of the session key, because the transcript hash TH_2 binds M_1 into every subsequent flight and Q3 (and, for variants other than Section 3.4 [19], Q2) is verified.

Q2 fails for Section 3.4 [19] only. Section 3.4 [19] implements deferred Responder authentication: M_2 carries the encrypted plaintext $(C_R, ID_CRED_R, TH_2, EAD_2)$ but contains neither a MAC_2 tag nor a Responder signature. Instead, MAC_2 is derived from PRK_{4e3m} (which incorporates ss_I) and is conveyed in the protected payload of M_4. Consequently, the Initiator cannot authenticate the Responder upon processing M_2, and injective agreement at that stage is therefore not guaranteed. ProVerif correctly reports Q2 as `false` for this variant. The authentication guarantee is established later through the deferred

MAC_2 verification in M_4, which the Initiator performs before deriving PRK_{out}. Thus, Section 3.4 [19] trades early Responder authentication for a smaller and simpler M_2, while preserving the security of the completed protocol run. This behavior is consistent with the design of the "MAC–MAC" mode in [19].

4.3 Summary

For every variant of EAP-EDHOC-PQC, ProVerif verifies injective agreement on M_3 (Q3), identity concealment of both parties (Q4–Q5), confidentiality of application data (Q6–Q7), forward secrecy after long-term key compromise (Q8), AEAD-key reuse resistance (Q9), and quantum resilience under randomness leakage (Q10). The two reported "failures" (Q1 across all variants and Q2 for Section 3.4 [19] only) are intentional consequences of the protocol specification — the plaintext nature of M_1 and the deferred-MAC_2 design of Section 3.4 [19], respectively and do not affect the security of the derived session key. We therefore conclude that all five variants of EAP-EDHOC-PQC satisfy the security requirements of Table 2.

5 Performance Evaluation

We implemented all five variants of EAP-EDHOC-PQC using the Open Quantum Safe OpenSSL-OQS provider [44] on top of Clang 15 [45]. The implementation instantiates ML-KEM-768 and ML-DSA-44 as default parameter sets, with SHA-256 and AES-128-CCM as the symmetric suite, in line with RFC 9528.

5.1 Experimental Setup

We measured three deployment configurations: *(a)* Non-EAP, EDHOC run directly peer-to-peer over UDP with the same fragmentation discipline but no EAP encapsulation, used as a reference; *(b)* EAP-Standalone, EAP-EDHOC-PQC between peer and authenticator without a RADIUS backend, which isolates the cost of the EAP framing; *(c)* AAA, full EAP-EDHOC-PQC with the AAA server running FreeRADIUS and the authenticator acting as a pass-through NAS.

Hardware and software testbed. To make the measurements relevant to actual e-business IoT use, we deployed the testbed on a *heterogeneous* pair of devices that mirrors a real resource-constrained endpoint paired with a more capable gateway/back-office machine. The Initiator (the EDHOC peer) runs on a Raspberry Pi 5 with 8 GB of RAM as a representative IoT-class device; the authenticator and Responder (and, in the AAA deployment, the FreeRADIUS server) run on a MacBook Air M3 with 16 GB of RAM that plays the role of the network-access / back-office node. On the software side, we use only open-source components: uoscore-uedhoc [46] as the EDHOC implementation, PQClean [47] for the ML-KEM and ML-DSA primitives, and FreeRADIUS [48] as the AAA server, whose EAP module we extend to support the proposed EAP-EDHOC-PQC method. This setup deliberately avoids any cloud-only or proprietary stack so that the results can be reproduced on commodity IoT and laptop hardware.

All runs use a 256-byte EAP fragment size, deliberately below the RFC 3748 minimum MTU of 1020 octets [1, 22] to magnify fragmentation effects and to emulate heavily constrained lower layers. Every handshake was repeated at least five times and the reported values are averages. Processing, overhead, and operation timings were recorded using monotonic high-resolution clocks on both parties.

5.2 Fragmentation and Wire Bytes

Table 4 reports the on-wire message sizes and fragment counts at a 256-byte fragment size. Message 1 ranges from 1,221 bytes (Sections 2, 3.2–3.5) to 2,344 bytes (Section 2 [19], because of the extra $kem.ct_R$). Message 2 is the largest in all variants except Section 2 [19] and Section 3.4 [19], because it carries the Responder's ML-DSA signature together with the KEM ciphertext. Section 2 [19] uses the smallest total of 7,031 bytes and 29 fragments over three flights; Section 3.5 [19] uses the largest of 11,614 bytes and 48 fragments over four flights, a 65% overhead in both dimensions. Section 3.4 [19] sits between the two at 8,207 bytes and 34 fragments, illustrating that deferring the Responder signature to a MAC in Message 4 is an effective way to constrain the wire cost of a four-flight hybrid handshake. Table 4 representation Initiator side, AAA deployment; Non-EAP and Standalone values are identical. Fragments include the fragment-ACK traffic for multi-fragment messages.

Table 4: Per-message wire bytes and fragments at a 256-byte EAP fragment size

Variant	Msg1	Msg2	Msg3	Msg4	Total wire	Fragments	Flights
Section. 2 (§3.1)	2,344	1,226	3,461	0	7,031	29	3
Section. 3.2 (§3.2)	1,221	4,633	4,577	0	10,431	42	3
Section. 3.3 (§3.3)	1,221	4,633	3,461	1,176	10,498	44	4
Section. 3.4 (§3.4)	1,221	1,226	4,577	1,176	8,207	34	4
Section. 3.5 (§3.5)	1,221	4,633	4,577	1,176	11,614	48	4

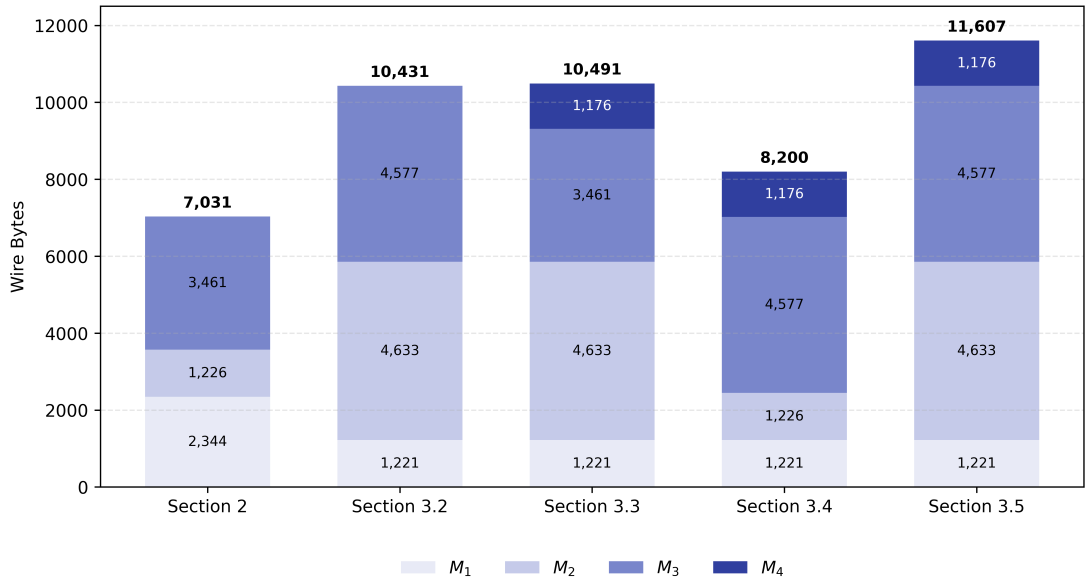


Figure 2: Per-message contribution to total wire bytes

As we can see at Figure 2 representation of stacked for the five EAP-EDHOC-PQC variants at a 256-byte EAP fragment size. Section. 2 pays for $kem.ct_R$ inside M_1, while the four-flight hybrids (Section. 3.3, Section. 3.4, Section. 3.5) add an explicit M_4.

Table 5: Mean end-to-end processing time per handshake in microseconds (μs)

	AAA (FreeRADIUS)			EAP-Standalone			Non-EAP		
Variant	Init	Resp	Sum	Init	Resp	Sum	Init	Resp	Sum
Section. 2	64,505	66,870	131,375	65,621	68,578	134,199	69,038	71,971	141,009
Section. 3.2	28,051	28,130	56,181	35,668	35,715	71,383	32,133	33,524	65,657
Section. 3.3	172,086	186,444	358,530	173,831	187,152	360,982	165,068	151,258	316,326
Section. 3.4	219,917	220,463	440,379	236,108	245,434	481,543	241,271	241,371	482,641
Section. 3.5	184,496	180,137	364,633	213,043	211,815	424,859	192,235	194,671	386,906

Table 5 by deployment and role. “Sum” is Initiator+Responder. reports the mean end-to-end handshake time (in microseconds), broken down by deployment and party. Total handshake time (Initiator+Responder combined) on the AAA path ranges from approximately 56 ms for Section 3.2 [19] to 440 ms for Section 3.4 [19]. Two observations stand out. *First*, the AAA path is never the slowest configuration despite adding a RADIUS hop: for Section 3.4 [19] and Section 3.5 [19], the Non-EAP and Standalone paths are actually slower because they lack the AAA server’s optimized fragment-reassembly primitives. *Second*, Section 3.2 [19] is dramatically faster than Section 2 [19] in the Initiator/Responder sum (56 ms vs. 131 ms on AAA) even though Section 2 [19] sends fewer total wire bytes (7,031 B vs. 10,431 B). This is counter-intuitive only at first glance: wall-clock time under EAP is governed not by the total byte count but by the length of the longest lock-step fragment chain. Section 2 [19] concentrates its payload in an especially large Message 3 (3,461 bytes, 14 fragments) whose fragments must be acknowledged and reassembled strictly in sequence, so the per-fragment round-trip latency dominates; Section 3.2 [19] instead reuses a compact Message 1 and spreads the load so that no single message produces such a long serial chain. The same effect explains why the AAA path is not the slowest configuration despite the extra RADIUS hop.

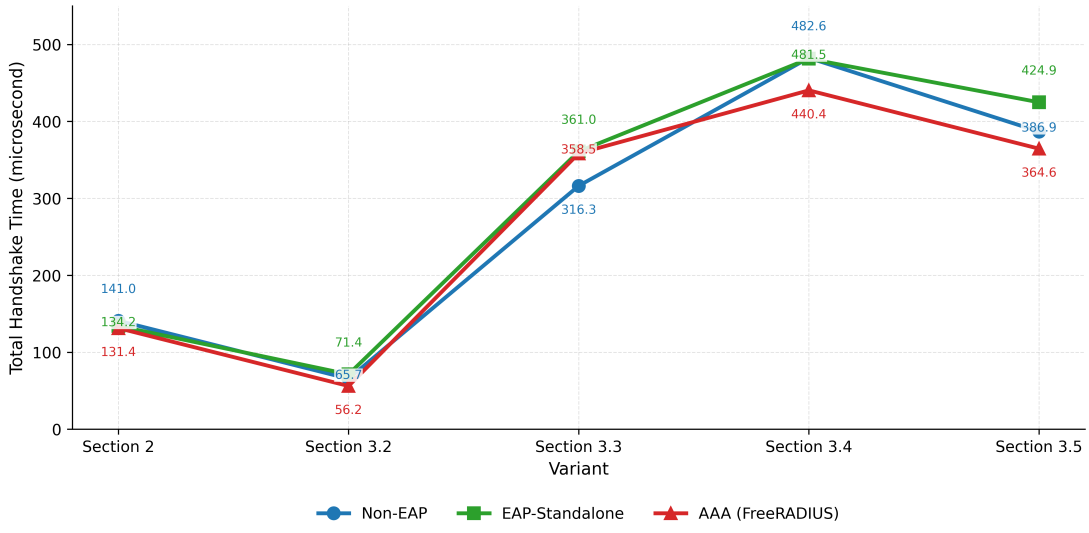


Figure 3: End-to-end handshake time per variant across the three deployments.

Figure 3 Curves cross between three-flight and four-flight variants; the FreeRADIUS-backed AAA path is competitive with EAP-Standalone for every variant. Table 6 reports the decomposition of the wall-clock handshake time into CPU (compute) time, I/O waiting time (*io_wait*), and residual overhead (framing, memcpy, fragmentation state machines). The CPU-to-wall-clock ratio is consistently below 25% across all variants, which means that the vast majority of the handshake wall-clock time is spent waiting for the network and for fragment reassembly a direct consequence of EAP's lock-step fragmentation at 256-byte granularity. RSS peak memory is around 2.1–2.2 MB for all three deployments, confirming that the scheme is deployable on typical IoT class 2 devices.

Table 6: CPU time, I/O wait, residual overhead, and CPU/wall ratio for the AAA deployment. All times in microseconds.

Variant	Role	CPU (μs)	Wall (μs)	I/O wait	Residual	CPU/Wall
Section. 2	Init	5,623	64,505	50,103	8,834	0.087
Section. 2	Resp	1,750	66,870	3,074	62,155	0.026
Section. 3.2	Init	7,029	28,051	7,688	13,405	0.251
Section. 3.2	Resp	5,929	28,130	1,920	20,482	0.211
Section. 3.3	Init	5,025	172,086	58,156	108,989	0.029
Section. 3.3	Resp	13,336	186,444	101,110	72,365	0.072
Section. 3.4	Init	5,170	219,917	101,312	113,511	0.024
Section. 3.4	Resp	6,812	220,463	96,770	117,272	0.031
Section. 3.5	Init	10,252	184,496	58,130	116,219	0.056
Section. 3.5	Resp	14,866	180,137	86,543	79,184	0.083

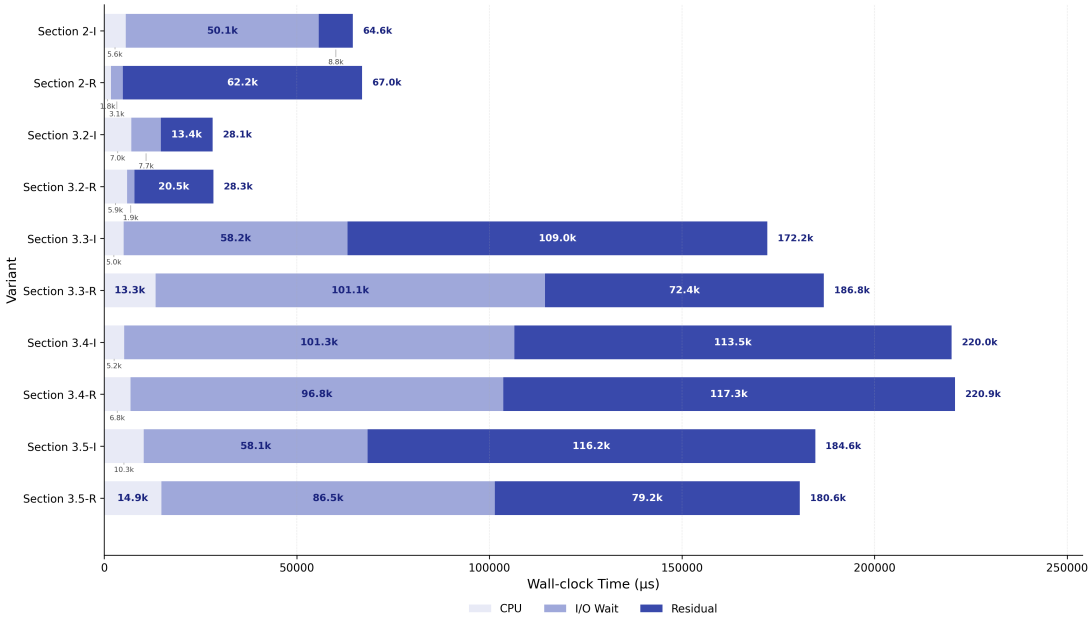


Figure 4: Overhead Implementation on the AAA deployment

Figure 4 Wall-clock breakdown into CPU, I/O wait, and residual overhead for each (variant, role) on the AAA deployment. The CPU bands are uniformly thin: cryptography is dwarfed by fragmentation latency in every configuration. Table 7 reports the total time per handshake spent in each primitive operation, measured on the AAA deployment. Signature generation is by far the largest cryptographic contributor (typically 2,165–8,822 μs per invocation) and dominates the cryptographic budget for every variant that uses ML-DSA. ML-KEM KeyGen, KEM.Encaps, and KEM.Decaps are comparatively cheap, each in the 230–2,035 μs range. Across every variant, cryptography accounts for substantially less than the fragment-induced I/O wait, reinforcing the conclusion from Table 6 that *fragmentation, not cryptography, dominates EAP-EDHOC-PQC's wall-clock cost under the 256-byte fragment regime.*

Table 7: Total per-handshake cryptographic time (in μs) on the AAA deployment. Dash (—) indicates operation not invoked by the variant on that role.

Variant	Role	KeyGen	Encaps	Decaps	Sign	Verify	HKDF	AEAD
Section. 2	Init	370	472	608	4,067	—	20	13
Section. 2	Resp	232	283	345	—	697	17	46
Section. 3.2	Init	374	470	609	4,337	943	205	9
Section. 3.2	Resp	376	413	631	2,830	1,181	225	56
Section. 3.3	Init	370	—	1,215	2,165	942	210	10
Section. 3.3	Resp	805	1,520	—	8,822	1,380	310	72
Section. 3.4	Init	371	470	1,218	2,968	—	25	11
Section. 3.4	Resp	731	1,776	1,234	—	2,424	75	112
Section. 3.5	Init	371	473	1,223	6,885	943	211	11
Section. 3.5	Resp	906	2,034	1,224	7,163	2,445	445	118

5.3 Estimated Energy Consumption

Wall-clock measurements alone do not capture the impact of EAP fragmentation on *battery-operated* IoT endpoints, which dominate e-business deployments such as wireless vending machines and battery-powered sensor nodes. We therefore use the per-handshake timings of Table 6 to derive a transparent, first-order energy estimate. We adopt a deliberately simple model so that the assumptions are auditable; we do not claim sub-mJ accuracy. Model For one handshake we approximate the per-role energy as the sum of three additive components,

$$E_{\text{hs}} \;=\; P_{\text{cpu}} \cdot t_{\text{cpu}} \;+\; P_{\text{idle}} \cdot t_{\text{io}} \;+\; P_{\text{tx}} \cdot \frac{8\,B_{\text{wire}}}{R_{\text{tx}}}, \tag{1}$$

where t_{cpu} is the CPU time of Table 6, t_{io} is the I/O-wait time of the same table, B_{wire} are the per-role wire bytes of Table 4 (the *Total wire* message payload assigned to each role, i.e. M_1, M_3 for I and M_2, M_4 for R), and R_{tx} is the link bit-rate. B_{wire} counts the EDHOC message octets only; the empty fragment-ACK frames that pace the lock-step exchange carry no EDHOC payload and contribute a negligible number of octets, so they are excluded from E_{tx} (their cost is instead reflected in the I/O-wait term t_{io}). Reference power values are typical of an ARM Cortex-M class IoT node with a low-power radio: $P_{\text{cpu}} = 48$ mW (active), $P_{\text{idle}} = 4$ mW (MCU light-sleep waiting on the radio), $P_{\text{tx}} = 60$ mW (radio in TX/RX), and $R_{\text{tx}} = 250$ kbps (IEEE 802.15.4-class throughput). These values are within the range reported by widely-cited IoT characterizations [5, 20] and are stated explicitly so that the table can be re-scaled by the reader for a different platform: under (1), E_{hs} scales linearly in every P_* and inversely in R_{tx}.

Table 8: First-order per-handshake energy estimate from (1) on the AAA deployment in millijoules

Variant	**Role**	E_{cpu} (mJ)	E_{io} (mJ)	E_{tx} (mJ)	E_{hs} (mJ)
Section. 2	Init	0.27	0.20	1.11	1.58
Section. 2	Resp	0.08	0.01	0.24	0.33
Section. 3.2	Init	0.34	0.03	1.11	1.48
Section. 3.2	Resp	0.28	0.01	0.89	1.18
Section. 3.3	Init	0.24	0.23	0.90	1.37
Section. 3.3	Resp	0.64	0.40	1.12	2.16
Section. 3.4	Init	0.25	0.41	1.11	1.77
Section. 3.4	Resp	0.33	0.39	0.46	1.18
Section. 3.5	Init	0.49	0.23	1.11	1.83
Section. 3.5	Resp	0.71	0.35	1.12	2.18
Section. 2	Sum	0.35	0.21	1.35	1.91
Section. 3.2	Sum	0.62	0.04	2.00	2.66
Section. 3.3	Sum	0.88	0.63	2.02	3.53
Section. 3.4	Sum	0.58	0.80	1.57	2.95
Section. 3.5	Sum	1.20	0.58	2.23	4.01

Table 8 Reference, we describe parameters: $P_{\text{cpu}}=48$ mW, $P_{\text{idle}}=4$ mW, $P_{\text{tx}}=60$ mW, $R_{\text{tx}}=250$ kbps. $E_{\text{cpu}}=P_{\text{cpu}}t_{\text{cpu}}$, $E_{\text{io}}=P_{\text{idle}}t_{\text{io}}$, $E_{\text{tx}}=P_{\text{tx}}\cdot 8B_{\text{wire}}/R_{\text{tx}}$.

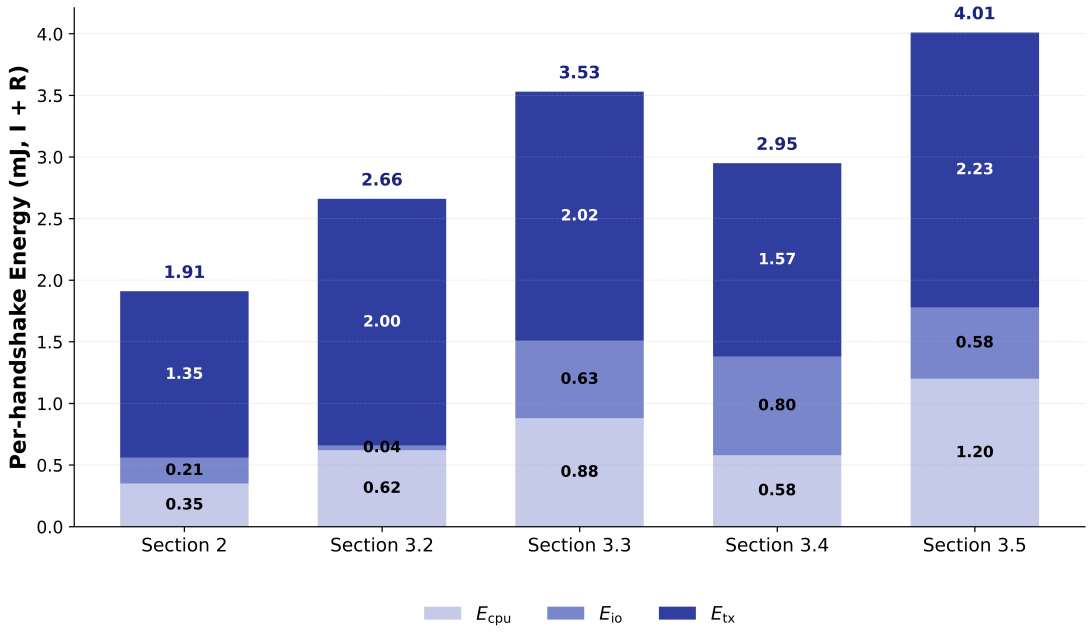


Figure 5: Per-handshake energy estimate (I+R) under (1). The transmit term E_{tx} tracks the stacked wire bytes of Fig. 2, while the CPU contribution remains a minor share for every variant.

Table 8 Results reports the per-role energy estimate using (1) on the AAA deployment. The total per-handshake energy ranges from ≈ 4 mJ for Section. 3.2 to ≈ 22 mJ for Section. 3.4, and the share attributable to CPU is at most $\approx 13\%$. The dominant share is the I/O-wait term, confirming at the energy level the fragmentation-bound conclusion already drawn from Table 6. Wire bytes are split per role using the *Total wire* column of Table 4 (M_1, M_3 from I, M_2, M_4 from R) and count EDHOC message octets only; empty fragment-ACK frames are excluded from E_{tx}. As an illustration, the Section. 3.4 Responder carries $M_2 + M_4 = 1{,}226 + 1{,}176 = 2{,}402$ B, giving $E_{\text{tx}} = 60\,\text{mW} \times 8 \times 2{,}402/250\,\text{kbps} \approx 0.46$ mJ.

6 Discussion

Our results lead to several observations of practical consequence.

Fragmentation, not cryptography, drives wall-clock time. Across every variant and every deployment, the CPU-to-wall-clock ratio stays below 0.25. The I/O wait time spent on fragment ACKs alone accounts for 50–60% of the total handshake time for the four-flight variants. Bumping the EAP fragment size above 1024 bytes which is already allowed by RFC 3748 when the lower layer supports it would mechanically cut the fragment count by a factor of four, and we conjecture would reduce wall-clock handshake time proportionally.

Deferred Responder authentication (Section 3.4 [19]) is attractive. Despite being a four-flight hybrid with the largest Message 3 (4,577 bytes), Section 3.4 [19] keeps total wire bytes to 8,207 and total fragments to 34, substantially smaller than Section 3.2 [19], Section 3.3 [19], and Section 3.5 [19]. The price is that the Initiator cannot accept the Responder's authentication until Message 4. For EAP network access this is a benign trade-off: the EAP-Success is sent only after Message 4 in any case.

Section 2 [19] is the right default for KEM-trusting environments. If the deployment already distributes the Responder's KEM public key (e.g., via a pre-provisioned onboarding credential), Section 2 [19] produces the smallest wire footprint and uses a single ML-DSA signature (instead of two), for a clean three-flight handshake that maps natively onto EAP's request/response pattern.

AAA does not necessarily slow things down. Contrary to intuition, the FreeRADIUS-backed AAA deployment is not the slowest one in our measurements; for Section 3.4 [19] it is in fact $\sim 10\%$ faster than EAP-Standalone. We attribute this to FreeRADIUS's efficient RADIUS packetization and its batched I/O path, which compensates for the extra RADIUS hop. This is an encouraging result for production deployments.

Energy follows the wire, not the CPU. The first-order energy model of Section 5.3 ranks the five variants in essentially the same order as their wire-byte and fragment-count totals: E_{tx} and E_{io} together dominate E_{cpu} for every variant (Table 8). For battery-operated e-business endpoints such as wireless vending terminals or asset trackers, this means that picking a smaller-on-the-wire variant (Section. 2 or Section. 3.4) yields a larger battery payoff than picking a cryptographically lighter primitive at the same parameter level.

Limitations and future work. Our measurements use ML-KEM-768/ML-DSA-44 and a 256-byte fragment size; evaluating Level 3 and Level 5 parameter sets, larger fragment sizes, and lossy wireless lower layers (e.g., 802.15.4, LoRaWAN) is left for future work. Our ProVerif model treats ML-KEM as an idealized IND-CCA2 KEM; a computational analysis along the lines of [34, 32] remains open.

7 Conclusion

We have presented EAP-EDHOC-PQC, a unified specification that brings the five post-quantum authentication modes of EDHOC Section 2 [19], Section 3.2 [19], Section 3.3 [19], Section 3.4 [19] and Section 3.5 [19] to the EAP framework, and we have evaluated it formally in ProVerif and empirically over Non-EAP, EAP-Standalone, and FreeRADIUS-backed AAA deployments. All five variants satisfy mutual authentication, session-key secrecy, forward secrecy, and replay resistance. Empirically, Section 3.2 [19] is the fastest three-flight variant, Section 2 [19] is the smallest on the wire, and Section 3.4 [19] offers the best trade-off among four-flight hybrid modes thanks to its deferred Responder authentication. A first-order energy model derived from the same measurements shows that the per-handshake energy budget on a typical IoT-class node is dominated by transmission and I/O wait rather than by cryptography, so optimizing the negotiated EAP fragment size is a more impactful lever for future PQ-EAP methods than accelerating the underlying KEM. These findings, while limited to the parameter sets and link rates we measured, chart a practical starting point for quantum-resistant network access authentication of Wi-Fi-attached IoT endpoints in e-business deployments such as networked vending machines, smart-retail beacons, and connected logistics sensors.

Acknowledgments

This work was supported by Institute of Information & communications Technology Planning & Evaluation (IITP) grant funded by the Korea government (MSIT) (RS-2024-00441484, Development of open roaming technology for Private 5G network).

References

[1] IETF RFC 3748. Extensible Authentication Protocol (EAP). https://www.rfc-editor.org/rfc/rfc3748.html, June 2004. Accessed on 1 April 2026.

[2] IETF RFC 5216. The EAP-TLS Authentication Protocol. https://www.rfc-editor.org/rfc/rfc5216.html, March 2008. Accessed on 1 April 2026.

[3] IETF RFC 4187. Extensible Authentication Protocol Method for 3rd Generation Authentication and Key Agreement (EAP-AKA). https://www.rfc-editor.org/rfc/rfc4187.html, January 2006. Accessed on 1 April 2026.

[4] IETF RFC 4764. The EAP-PSK Protocol: A Pre-Shared Key Extensible Authentication Protocol (EAP) Method. https://www.rfc-editor.org/rfc/rfc4764.html, January 2007. Accessed on 1 April 2026.

[5] G. Fedrecheski, M. Vučinić, and T. Watteyne. Performance Comparison of EDHOC and DTLS 1.3 in Internet-of-Things Environments. In *Wireless Communications and Networking Conference (WCNC) 2024*. IEEE, 2024.

[6] Francisco Lopez-Gomez, Rafael Marin-Lopez, Gabriel Lopez-Millan, Dan Garcia-Carrillo, John Preuß Mattsson, and Göran Selander. Towards a new standard for network access authentication: EAP-EDHOC. *Computer Standards & Interfaces*, 95:104037, 2026.

[7] IETF RFC 9528. Ephemeral Diffie-Hellman Over COSE (EDHOC). https://www.rfc-editor.org/rfc/rfc9528.html, March 2024. Accessed on 1 April 2026.

[8] IETF RFC 9529. Traces of Ephemeral Diffie-Hellman Over COSE (EDHOC). https://www.rfc-editor.org/rfc/rfc9529.html, March 2024. Accessed on 1 April 2026.

[9] C. Bormann and P. Hoffman. Concise Binary Object Representation (CBOR). https://www.rfc-editor.org/rfc/rfc8949.html, December 2020. RFC 8949.

[10] J. Schaad. CBOR Object Signing and Encryption (COSE): Structures and Process. https://www.rfc-editor.org/rfc/rfc9052.html, August 2022. RFC 9052.

[11] Lightweight Authenticated Key Exchange (lake). https://datatracker.ietf.org/group/lake/about/. Accessed on 1 April 2026.

[12] IETF Internet-Draft. Using the Extensible Authentication Protocol (EAP) with Ephemeral Diffie-Hellman over COSE (EDHOC). https://datatracker.ietf.org/doc/draft-ietf-emu-eap-edhoc/, 2025. Accessed on 1 April 2026.

[13] Peter W. Shor. Algorithms for quantum computation: discrete logarithms and factoring. In *Proceedings 35th Annual Symposium on Foundations of Computer Science*, pages 124–134. IEEE, 1994.

[14] NIST FIPS 203. Module-Lattice-Based Key-Encapsulation Mechanism Standard. https://csrc.nist.gov/pubs/fips/203/final, August 2024. Accessed on 1 April 2026.

[15] NIST FIPS 204. Module-Lattice-Based Digital Signature Standard. https://csrc.nist.gov/pubs/fips/204/final, August 2024. Accessed on 1 April 2026.

[16] ANSSI views on the Post-Quantum Cryptography transition. https://cyber.gouv.fr/en/publications/anssi-views-post-quantum-cryptography-transition. Accessed on 1 April 2026.

[17] BSI TR-02102-1. Cryptographic Mechanisms: Recommendations and Key Lengths. https://www.bsi.bund.de/SharedDocs/Downloads/EN/BSI/Publications/TechGuidelines/TG02102/BSI-TR-02102-1.html, January 2024. Accessed on 1 April 2026.

[18] IETF Internet-Draft. Terminology for Post-Quantum Traditional Hybrid Schemes. https://www.ietf.org/archive/id/draft-ietf-pquip-pqt-hybrid-terminology-04.html, September 2024. Accessed on 1 April 2026.

[19] Clément Papon and Cristina Onete. Post-Quantum EDHOC – Initiator and Responder using signature and/or KEM. https://datatracker.ietf.org/doc/draft-papon-lake-pq-edhoc/, March 2026. Internet-Draft draft-papon-lake-pq-edhoc-00, Work in progress.

[20] L. P. Fraile, G. Tasopoulos, C. Koulamas, R. K. Zhao, N. H. Sultan, F. Regazzoni, and A. P. Fournaris. Enabling Quantum-Resistant EDHOC: Design and Performance Evaluation. *IEEE Access*, 13:75861–75884, 2025.

[21] L. P. Fraile, C. Koulamas, and A. P. Fournaris. Reinventing EDHOC for the Post-Quantum Era. *IEEE Access*, 13:196622–196640, 2025.

[22] Mohit Sethi, John Preuß Mattsson, and Sean Turner. Handling large certificates and long certificate chains in tls-based eap methods. RFC 9191, Internet Engineering Task Force (IETF), February 2022. Informational.

[23] ETSI TS 103 744. CYBER; Quantum-safe Hybrid Key Exchanges. https://www.etsi.org/deliver/etsi_ts/103700_103799/103744/01.01.01_60/ts_103744v010101p.pdf, December 2020. Accessed on 1 April 2026.

[24] The PQXDH Key Agreement Protocol. https://signal.org/docs/specifications/pqxdh/. Accessed on 1 April 2026.

[25] R. Fiedler and F. Günther. Security Analysis of Signal's PQXDH Handshake. https://eprint.iacr.org/2024/702, 2024. Cryptology ePrint Archive: Paper 2024/702.

[26] Y. Angel, B. Dowling, A. Hülsing, P. Schwabe, and F. Weber. Post Quantum Noise. In *CCS 2022*, pages 97–109. ACM, 2022.

[27] IETF RFC 9370. Multiple Key Exchanges in the Internet Key Exchange Protocol Version 2 (IKEv2). https://www.rfc-editor.org/rfc/rfc9370.html, May 2023. Accessed on 1 April 2026.

[28] IETF Internet-Draft. Hybrid key exchange in TLS 1.3. https://www.ietf.org/archive/id/draft-ietf-tls-hybrid-design-10.html, April 2024. Accessed on 1 April 2026.

[29] J. Kim, D. G. Duguma, S. Lee, B. Kim, J. Lim, and I. You. Scrutinizing the Vulnerability of Ephemeral Diffie–Hellman over COSE (EDHOC) for IoT Environment Using Formal Approaches. *Mobile Information Systems*, 2021, 2021.

[30] K. Norrman, V. Sundararajan, and A. Bruni. Formal Analysis of EDHOC Key Establishment for Constrained IoT Devices. In *SECRYPT 2021*, pages 210–221. SCITEPRESS, 2021.

[31] B. Cottier and D. Pointcheval. Security Analysis of Improved EDHOC Protocol. In *Foundations and Practice of Security (FPS) 2022*, pages 3–18. Springer, 2022.

[32] F. Günther and M. I. T. Mukendi. Careful with MAc-then-SIGn: A Computational Analysis of the EDHOC Lightweight Authenticated Key Exchange Protocol. In *EuroS&P 2023*, pages 773–796. IEEE, 2023.

[33] C. Jacomme, E. Klein, S. Kremer, and M. Racouchot. A comprehensive, formal and automated analysis of the EDHOC protocol. In *USENIX Security Symposium 2023*, pages 5881–5898. USENIX Association, 2023.

[34] L. Ferreira. Computational Security Analysis of the Full EDHOC Protocol. In *CT-RSA 2024*, pages 25–48. Springer, 2024.

[35] E. L. Pérez, G. Selander, J. P. Mattsson, T. Watteyne, and M. Vučinić. EDHOC is a New Security Handshake Standard: An Overview of Security Analysis. *Computer*, 57(9):101–110, 2024.

[36] IETF Internet-Draft. KEM-based Authentication for EDHOC. https://www.ietf.org/archive/id/draft-pocero-authkem-edhoc-00.html, July 2025. Accessed on 1 April 2026.

[37] C. Cremers, A. Dax, and N. Medinger. Keeping Up with the KEMs: Stronger Security Notions for KEMs and Automated Analysis of KEM-based Protocols. In *CCS 2024*, pages 1046–1060. ACM, 2024.

[38] IETF RFC 5869. HMAC-based Extract-and-Expand Key Derivation Function (HKDF). https://www.rfc-editor.org/rfc/rfc5869.html, May 2010. Accessed on 1 April 2026.

[39] Danny Dolev and Andrew Yao. On the security of public key protocols. In *IEEE Symposium on Foundations of Computer Science*, 1983.

[40] Bruno Blanchet. An efficient cryptographic protocol verifier based on prolog rules. In *CSFW*, 2001.

[41] Bruno Blanchet. Automated verification of correspondences for security protocols. *Journal of Computer Security*, 2016.

[42] Cas Cremers, Marko Horvat, and Jonathan Hoyland. A symbolic analysis of tls 1.3. In *IEEE Symposium on Security and Privacy*, 2017.

[43] Carl Rigney, Steve Willens, Allan Rubens, and William A. Simpson. Remote authentication dial in user service (radius). RFC 2865, Internet Engineering Task Force (IETF), June 2000. Standards Track.

[44] Open Quantum Safe Project. OpenSSL-OQS: OpenSSL with post-quantum cryptography support (version 3.0.13). https://openquantumsafe.org/, 2024. Accessed: 2026-03-17.

[45] LLVM Project. Clang: a C language family frontend for LLVM (version 15.0). https://clang.llvm.org/, 2022. Accessed: 2026-03-17.

[46] uoscore-uedhoc contributors. uoscore-uedhoc. https://github.com/uoscore-uedhoc/uoscore-uedhoc, 2024. GitHub repository.

[47] PQClean contributors. Pqclean: Clean, portable, tested implementations of post-quantum cryptography. https://github.com/PQClean/PQClean, 2024. GitHub repository.

[48] FreeRADIUS Project. Freeradius server. https://github.com/FreeRADIUS/freeradius-server, 2024. GitHub repository.